# DIY Spotify Mini Player — ESP32 + OLED + Buttons

Full build guide: hardware, wiring, Spotify API setup, and complete Arduino code with Previous / Play-Pause / Next buttons.

---

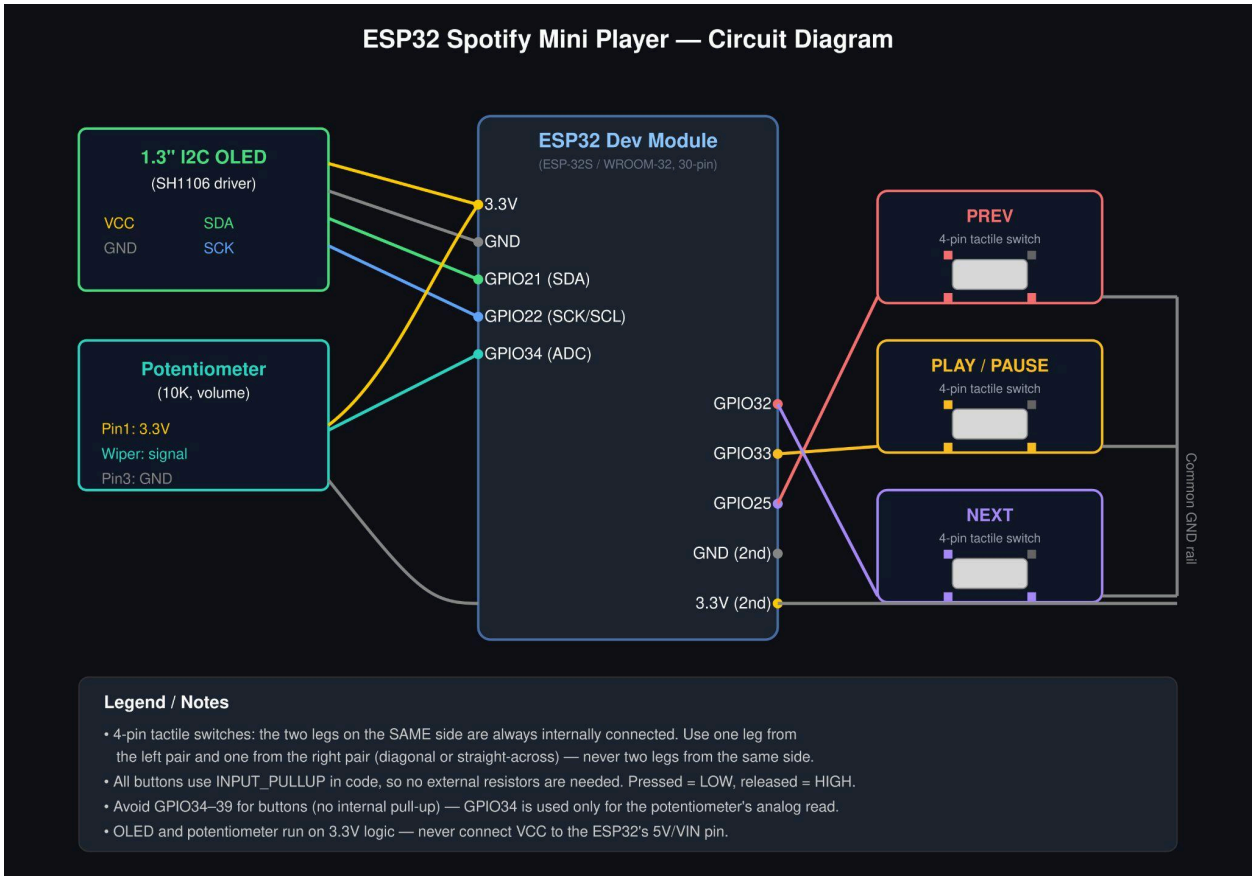
## 1. Parts List

| **Part** | **Notes** |
|---|---|
| ESP32 DevKit (any variant) | Needs Wi-Fi |
| 1.3" I2C OLED (SH1106 driver) | 4-pin: VCC, GND, SCL, SDA |
| 3x momentary push buttons | Previous, Play/Pause, Next |
| 10K potentiometer | Volume control |
| Breadboard | |
| Jumper wires | |
| USB cable | Matches your ESP32's port |

No resistors needed for the buttons — the code uses the ESP32's internal pull-up resistors. No resistors needed for the potentiometer either.

---

# 2. Wiring




**OLED (I2C):**

- VCC → 3.3V
- GND → GND
- SDA → GPIO21
- SCL/SDK → GPIO22

**Buttons — 4-pin tactile switches (each: one leg to the GPIO, the diagonal-opposite leg to GND):**

- Previous → GPIO32
- Play/Pause → GPIO33
- Next → GPIO25

4-pin tactile switches have two internal pairs: the two legs on the same side (e.g. top-left + bottom-left) are always connected to each other, even unpressed. Pressing the button bridges the two sides together. So wire **one leg from the left side** to your GPIO pin and **one leg from**

**the right side** to GND — never two legs from the same side, or the button will read as permanently pressed. If you're unsure which legs are paired, check with a multimeter's continuity mode, or just try it: an always-LOW reading means you picked two legs from the same side.

**Potentiometer (volume):**

- Outer pin 1 → 3.3V
- Outer pin 2 → GND
- Middle pin (wiper) → GPIO34

⚠️ Avoid GPIO34–39 for the buttons — those pins are input-only and have no internal pull-up resistor on the ESP32. GPIO34 is fine (in fact ideal) for the potentiometer since it's a pure analog read with no pull-up needed, and it sits on ADC1, which stays accurate even while Wi-Fi is active (ADC2 pins can get flaky when Wi-Fi is on).

---

## 3. Spotify Developer Setup

1. Go to the Spotify Developer Dashboard and log in.
2. Click **Create App**. Fill in a name/description.
3. Set **Redirect URI** to `https://example.com/callback` (any placeholder URL works — you just need it to complete the auth flow once).
4. Save, then copy your **Client ID** and **Client Secret**.
5. Under **Users and Access**, add your own Spotify account as a user (required while the app is in Development Mode).

### Get a refresh token (one-time step, done on your computer, not the ESP32)

The ESP32 can't do a browser-based OAuth login, so you generate a **refresh token** once on your PC and hardcode it into the sketch.

1. Build this URL, replacing `YOUR_CLIENT_ID` and `YOUR_REDIRECT_URI` (URL-encoded):

   https://accounts.spotify.com/authorize?response_type=code&client_id=YOUR_CLIENT_ID&scope=user-read-currently-playing%20user-read-playback-state%20user-modify-playback-state&redirect_uri=YOUR_REDIRECT_URI

2. Open it in a browser, log in, approve access. You'll be redirected to your redirect URI with a `?code=...` parameter in the address bar — copy that `code`.
3. Exchange the code for tokens (run this locally, e.g. via `curl` or Postman):

curl -X POST https://accounts.spotify.com/api/token

-d grant_type=authorization_code

-d code=YOUR_CODE

-d redirect_uri=YOUR_REDIRECT_URI

-d client_id=YOUR_CLIENT_ID

-d client_secret=YOUR_CLIENT_SECRET

4. The JSON response includes `refresh_token` — save it. This is what goes in the sketch. It doesn't expire under normal use (only if revoked).

**Note:** Controlling playback (play/pause/skip) requires a **Spotify Premium** account — the Web API playback-control endpoints reject Free accounts.

---

## 4. Arduino IDE Setup

1. Install Arduino IDE (2.x recommended).
2. File → Preferences → Additional Board Manager URLs, add:

   https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

3. Tools → Board → Boards Manager → search "esp32" → install.
4. Select your board (e.g. "ESP32 Dev Module") and the correct COM port.
5. Install libraries via Library Manager (Tools → Manage Libraries):
   - `Adafruit SH110X` (driver for the 1.3" SH1106 OLED)
   - `Adafruit GFX Library`
   - `ArduinoJson` (for parsing Spotify's API responses)

---

## 5. Upload & Test

1. Wire everything per the diagram, double-check OLED is on 3.3V (not 5V).
2. Plug in the ESP32, select the right port in Arduino IDE.
3. Upload the sketch.

4. Open Serial Monitor (115200 baud) to watch for Wi-Fi connect and token refresh logs.
5. Play something on Spotify (on your phone/PC, same account) — the OLED should show track/artist/progress within ~5 seconds.
6. Press the buttons — Previous/Next should skip, Play/Pause should toggle.

### Troubleshooting

- **Blank OLED**: check I2C address — most SH1106 1.3" modules use `0x3C`, but some use `0x3D`. Run an I2C scanner sketch if unsure.
- **401 errors**: refresh token invalid/expired — regenerate it via the OAuth steps.
- **403 on play/pause/skip**: account isn't Premium, or no active Spotify device is open.
- **Buttons not responding**: confirm wiring goes to GND, not 3.3V, and that pins aren't 34–39.
- **Volume not changing / jittery**: cheap potentiometers can have noisy wipers — the `VOLUME_DEADZONE` (2%) filters small jitter. Increase it to 3–5 if it's still twitchy. Also note volume control only works on devices that report `volume_percent` support (some, like the web player, don't).

---

## 6. Optional Upgrades

- Add a 4th button for shuffle or repeat toggle.
- Show the current volume percentage on the OLED next to the progress bar.
- Use `esp_sleep` to dim/sleep the OLED after inactivity.
- Cache album art bitmap and show it (needs more RAM — ESP32 handles it fine, SSD1306 is monochrome only though, so you'd dither it).